JobPilot: Smart Job Matcher

BAX-423 Final Project Technical Brief

1. Executive Summary

JobPilot is an end-to-end job recommendation application for MSBA-style job search workflows. It combines an offline LinkedIn/Kaggle snapshot with live Adzuna ingestion, extracts candidate profiles from resumes or structured inputs, retrieves jobs with dense embeddings, applies hard constraints, re-ranks candidates with explainable scores, learns from accept/reject/skip feedback, and generates tailored Markdown resumes for selected roles.

The app is built in Streamlit with a SQLite persistence layer and FAISS vector index. The core workflow is:

Kaggle + Adzuna -> deduped SQLite jobs -> resume/manual profile -> FAISS recall -> hard filters -> weighted re-ranking -> feedback adaptation -> resume/export

2. Architecture And Pipeline Design

Data ingestion. The cold-start dataset is a cleaned 30,000-row LinkedIn/Kaggle job snapshot stored under data/. For current postings, the sidebar Fetch New Jobs action calls the Adzuna API, normalizes JSON into the shared schema, deduplicates against existing SQLite job IDs, inserts only new rows, and adds those new rows to the in-memory FAISS index for the current session. A Kafka-style StreamingPipeline class with queue-based producer/consumer logic is also implemented in engine/ingestion.py, while the UI uses a synchronous one-shot fetch for reliable demos.

Profile intake. Users can upload a PDF/DOCX resume or enter structured background, target roles, skills, location, salary, visa need, and dealbreakers manually. PDF/DOCX text is extracted with PyPDF2/python-docx, then parsed with Gemini (gemini-flash-latest) into structured fields. Manual inputs are merged with extracted resume fields, so the app can be tested without a resume file.

Retrieval and storage. Jobs and candidate queries are embedded using sentence-transformers/all-MiniLM-L6-v2 into 384-dimensional normalized vectors. FAISS IndexFlatIP performs cosine-similarity candidate generation. The app retrieves a larger candidate pool before filtering to avoid starving results when salary, location, and seniority constraints are strict.

3. BAX-423 Techniques

Technique 1: Dense Vector Retrieval (Embeddings / Semantic Search). A BM25-style keyword match struggles with vocabulary gaps such as "PyTorch" vs. "deep learning." JobPilot instead embeds job titles, skills, locations, and descriptions into dense vectors and uses FAISS recall to find semantically related jobs. This is the L1 candidate generation step.

Technique 2: Multi-Stage Ranking. Retrieval alone cannot enforce dealbreakers. JobPilot uses a funnel:
1. Candidate generation: FAISS recalls top candidates.
2. Hard filtering: deterministic rules remove jobs violating dealbreakers, salary, location, visa constraints, seniority limits, contract/unpaid terms, and persona-specific risk words.
3. Re-ranking: surviving jobs receive an explainable weighted score:

Score = 0.25*Semantic + 0.25*Skill + 0.25*Role + 0.10*Location + 0.15*Feedback

Technique 3: Adaptive Feedback. User reactions are stored as accept/reject/skip events. Accept gives positive weight, reject gives stronger negative weight, and skip is neutral. The re-ranker propagates feedback to related companies, titles, and experience levels so repeated rejection of a senior or mid-senior role lowers similar jobs in future rankings.

4. Evaluation And Test Personas

The Benchmark tab reports Precision@10, NDCG@10, and a deterministic Pass Criteria check for the four required personas. BM25 is the sparse baseline, embedding-only FAISS is the L5

semantic baseline, and the full multi-stage system is the L7 ranking pipeline.

Persona; Inputs Tested; Deterministic Pass Criteria Checked In Code; Result
Aisha, ML career pivoter; Data Analyst, Python/SQL/pandas/sklearn/PyTorch, ML Engineer/Applied Scientist/Data Scientist, Bay Area/remote, salary >= 140k, no Senior/Staff/Defense/Military; Top-10 has no senior/staff/defense/military wording and every row has ML/AI/data-science signal; PASS
Marcus, new grad; Recent MSBA, Python/R/SQL/Tableau/PySpark, Data Analyst/BI/Junior DS/Analytics Engineer, salary >= 80k, no 3+ years/contract/unpaid; Top-10 has no 3+ year, senior, contract, or unpaid wording; PASS
Priya, ML infrastructure; Senior SWE, Java/Python/Kubernetes/Kafka/Spark/TensorFlow/AWS, MLOps/ML Platform/Senior ML Engineer, NYC/remote, salary >= 200k, no Junior/Entry; Top-10 has no junior/entry wording and contains ML infrastructure signals such as Kafka, Spark, Kubernetes, MLOps, platform, or AWS; PASS
Kenji, visa-constrained; International CS student, Python/C++/PyTorch/NLP/CV, Research Scientist/ML Engineer/Applied Scientist/AI Engineer, US, salary >= 120k, no contract/temp/1099, visa required; Top-10 has no contract/temp/1099 or explicit no-sponsorship wording; known sponsor/research-lab terms receive ranking support; PASS

The app also includes live online metrics above the job list: average Top-10 relevance, session hit rate from accept/reject feedback, and Top-10 skill coverage.

5. Limitations

Sponsor and company-size data are imperfect. The dataset does not provide reliable company headcount or guaranteed H-1B sponsorship fields, so JobPilot uses negative sponsorship filters and known-sponsor/research-lab heuristics. A production system should enrich companies from H-1B disclosure and firmographic datasets.

Experience requirements use text heuristics. The code scans titles, experience levels, and descriptions for seniority and year-requirement patterns. It catches common cases such as "3+ years," "mid-senior," and "senior," but unusual wording can still slip through.

Embedding truncation limits context. all-MiniLM-L6-v2 is fast but short-context. Long job descriptions may hide important constraints past the indexed text. A future version should chunk descriptions and pool scores across chunks.

Resume generation can over-expand sparse inputs. If a user provides only one sentence of background, Gemini may create plausible but overly detailed resume bullets. The safest workflow is to upload a complete resume and review the generated Markdown before use.